System Identification (Part 2)

Prediction Error Methods, Polynomial Models, Neural Networks

Assoc. Prof. Karl Ezra Pilario
Process Systems Engineering Laboratory
Department of Chemical Engineering

© 2023 Karl Ezra S. Pilario, PhD.

Outline

- Recall: System Identification

- General Linear Model Structure
  - SISO Models: AR, ARX, ARMA, ARMAX
  - N-step Ahead Prediction vs. Simulation

- Nonlinear Model Structures
  - Hammerstein-Wiener Process
  - Recurrent Neural Networks in MATLAB
  - Application to Simulated Data

© 2023 Karl Ezra S. Pilario, PhD.

System Identification

- Building models from real-world process data.
- In the context of state-space models, system identification refers to the problem of finding $\boldsymbol{A}, \boldsymbol{B}, \boldsymbol{C}, \boldsymbol{D}$ that best fits the process data.

Algorithm:

Subspace Identification Methods (**SIM**)

Training data $\{\boldsymbol{u}_i, \boldsymbol{y}_i\}$

Matrix projections

State sequence, $\boldsymbol{x}_i$

Multivariate Regression

Final Model

Prediction Error Methods (**PEM**)

Training data $\{\boldsymbol{u}_i, \boldsymbol{y}_i\}$

Initialize model parameters

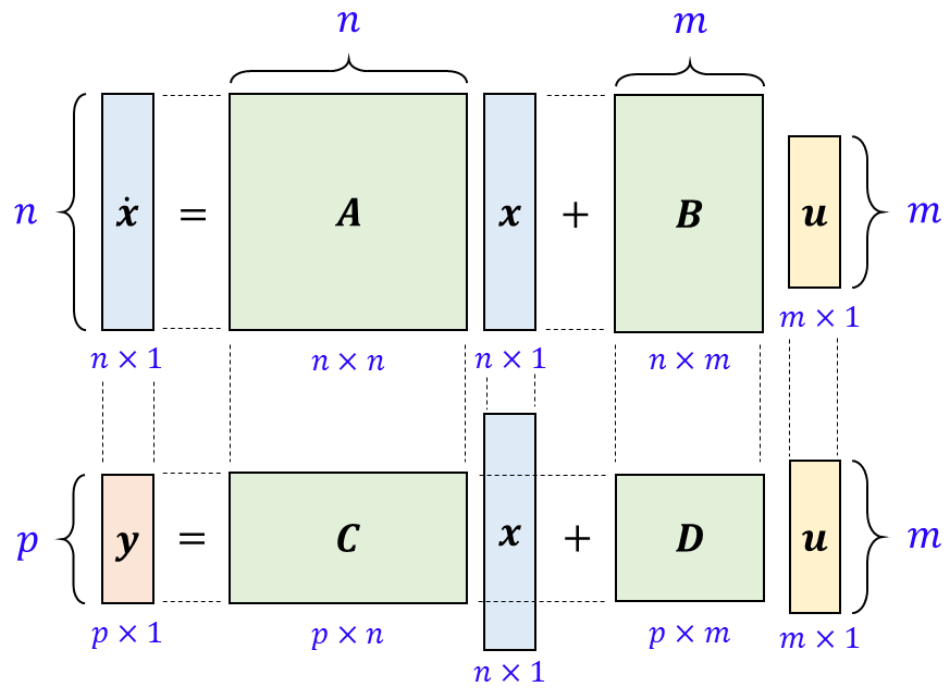
Model

Iterate

Final Model

State-space Model

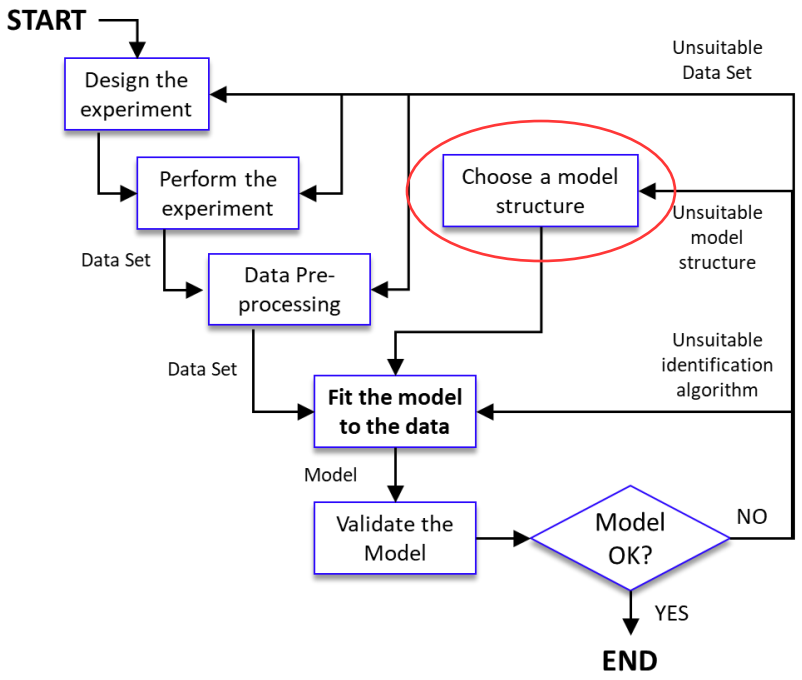
$$\dot{\boldsymbol{x}} = \boldsymbol{A}\boldsymbol{x} + \boldsymbol{B}\boldsymbol{u}$$
$$\boldsymbol{y} = \boldsymbol{C}\boldsymbol{x} + \boldsymbol{D}\boldsymbol{u}$$



© 2023 Karl Ezra S. Pilario, PhD.

Other Model Structures

General System Identification Cycle:
(Verhaegen and Verdult, 2007)

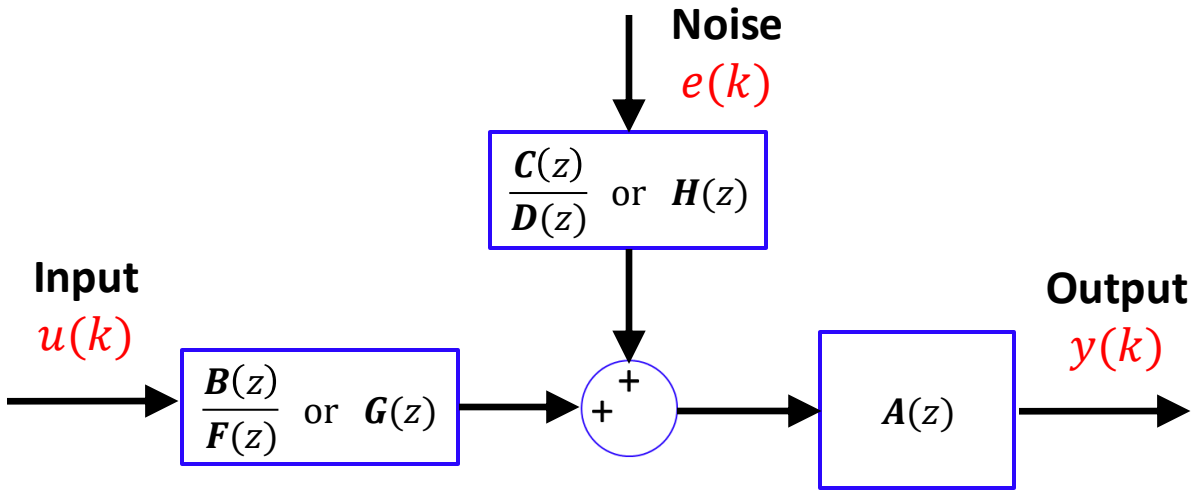


If the goal of system identification is for **prediction or simulation**, surely there are models **simpler than state-space models** that can fulfill the same purpose.

- State-space model structures are used because we are sure that our process is governed by some set of DAEs.
- We can consider a **generalized linear model** of the form:

$$\boldsymbol{A}(z)y(k) = \frac{\boldsymbol{B}(z)}{\boldsymbol{F}(z)}u(k) + \frac{\boldsymbol{C}(z)}{\boldsymbol{D}(z)}e(k)$$

where **A**, **B**, **C**, **D**, **F** are *polynomials* in z, and z denotes the forward shift operator: $z^{-1}y(k) = y(k-1)$.



© 2023 Karl Ezra S. Pilario, PhD.

Generalized Linear Models

- Four common SISO models: AR, ARX, ARMA, ARMAX

- **AR**, Autoregressive Model

$$\boldsymbol{A}(z)y(k) = \frac{\boldsymbol{B}(z)}{\boldsymbol{F}(z)}u(k) + \frac{\boldsymbol{C}(z)}{\boldsymbol{D}(z)}e(k) \Rightarrow \boldsymbol{A}(z)y(k) = e(k)$$

$$\boldsymbol{A}(z)y(k) = e(k)$$
$$(z^0 + a_0 + a_1 z^{-1} + a_2 z^{-2} + \cdots)y(k) = e(k)$$
$$y(k) + a_0 + a_1 y(k-1) + a_2 y(k-2) + \cdots = e(k)$$
$$\boxed{y(k) = a_0 + \sum_{i=1}^{p} a_i y(k-i) + e(k)} \quad AR(p)$$

- **ARX**, Autoregressive with Exogeneous Input Model

$$\boldsymbol{A}(z)y(k) = \frac{\boldsymbol{B}(z)}{\boldsymbol{F}(z)}u(k) + \frac{\boldsymbol{C}(z)}{\boldsymbol{D}(z)}e(k) \Rightarrow \boldsymbol{A}(z)y(k) = \boldsymbol{B}(z)u(k) + e(k)$$

$$\boxed{y(k) = a_0 + \sum_{i=1}^{p} a_i y(k-i) + \sum_{j=1}^{q} b_j u(k-j) + e(k)} \quad ARX(p,q)$$

- **ARMA**, Autoregressive with Moving Average Model

$$\boldsymbol{A}(z)y(k) = \frac{\boldsymbol{B}(z)}{\boldsymbol{F}(z)}u(k) + \frac{\boldsymbol{C}(z)}{\boldsymbol{D}(z)}e(k) \Rightarrow \boldsymbol{A}(z)y(k) = \boldsymbol{C}(z)e(k)$$

$$\boxed{y(k) = a_0 + \sum_{i=1}^{p} a_i y(k-i) + \sum_{j=1}^{q} c_j e(k-j) + e(k)} \quad ARMA(p,q)$$

- **ARMAX**, ARMA with Exogeneous Input Model

$$\boldsymbol{A}(z)y(k) = \frac{\boldsymbol{B}(z)}{\boldsymbol{F}(z)}u(k) + \frac{\boldsymbol{C}(z)}{\boldsymbol{D}(z)}e(k)$$

$$\Rightarrow \boldsymbol{A}(z)y(k) = \boldsymbol{B}(z)u(k) + \boldsymbol{C}(z)e(k)$$

$$\boxed{y(k) = a_0 + \sum_{i=1}^{p} a_i y(k-i) + \sum_{j=1}^{q} b_j u(k-j) + \sum_{l=1}^{r} c_l e(k-l) + e(k)}$$

$$ARMAX(p,q,r)$$

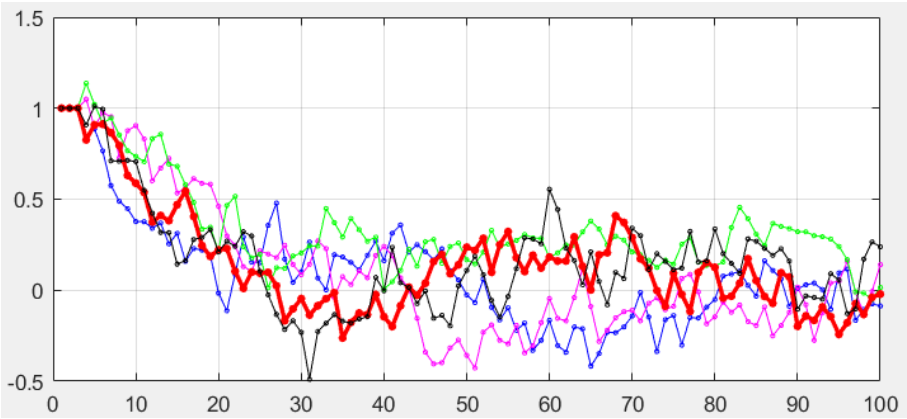
© 2023 Karl Ezra S. Pilario, PhD.

Generalized Linear Models

- Samples from an $\boldsymbol{AR(3)}$ **process:**

$$y(k) = 0.8\, y(k-1) - 0.2\, y(k-2) + 0.3\, y(k-3) + e(k)$$

where $e(k) \sim \mathcal{N}(0, 0.1^2)$ and $y(1) = y(2) = y(3) = 1$



First few values of the red time series:

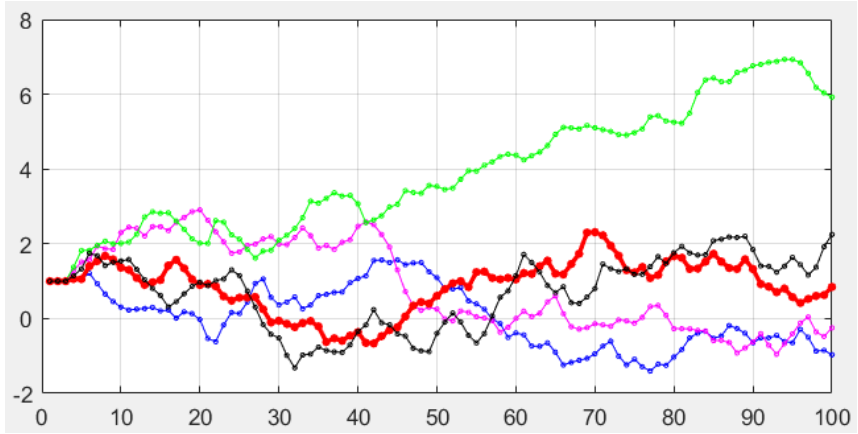
k	1	2	3	4	5	6	...
$y(k)$	1.00	1.00	1.00	0.826	0.909	0.912	...

7	8	9	10	11	12	...
0.865	0.796	0.631	0.588	0.537	0.375	...

- Samples from an $\boldsymbol{ARMA(2,1)}$ **process:**

$$y(k) = 0.9\, y(k-1) + 0.1\, y(k-2) + e(k) + 2.0\, e(k-1)$$

where $e(k) \sim \mathcal{N}(0, 0.1^2)$ and $y(1) = y(2) = y(3) = 1$



First few values of the red time series:

k	1	2	3	4	5	6	...
$y(k)$	1.00	1.00	1.00	1.062	1.056	1.402	...

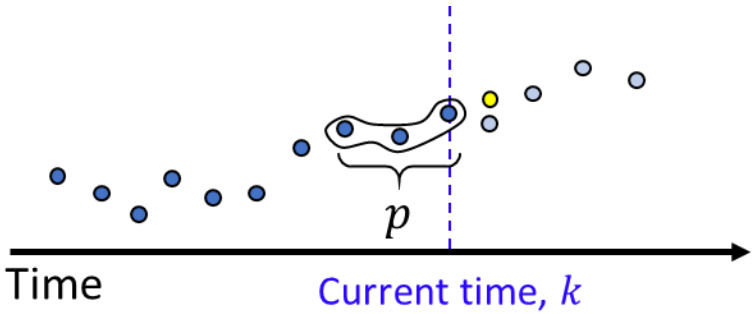
7	8	9	10	11	12	...
1.538	1.677	1.583	1.362	1.304	1.091	...

© 2023 Karl Ezra S. Pilario, PhD.

Least-squares Estimation

- **AR**, Autoregressive Model

$$y(k) = a_0 + \sum_{i=1}^{p} a_i y(k-i) + e(k)$$



How to find $a_0, a_1, a_2, \ldots a_p$?

Let:

$$\boldsymbol{Y} = \begin{bmatrix} y(p+1) \\ y(p+2) \\ y(p+3) \\ \vdots \\ y(N) \end{bmatrix}$$

where:

$\boldsymbol{Y} \in \Re^{(N-p)\times 1}$ = targets
$\boldsymbol{W} \in \Re^{(p+1)\times 1}$ = weights
$\boldsymbol{H} \in \Re^{(N-p)\times(p+1)}$ = regressors

$$\boldsymbol{H} = \begin{bmatrix} 1 & y(p) & y(p-1) & y(p-2) & \ldots & y(1) \\ 1 & y(p+1) & y(p) & y(p-1) & \ldots & y(2) \\ 1 & y(p+2) & y(p+1) & y(p) & \ldots & y(3) \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & y(N-1) & y(N-2) & y(N-3) & \cdots & y(N-p) \end{bmatrix} \quad \boldsymbol{W} = \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_p \end{bmatrix}$$

The problem becomes: Find **W** such that

$$\min_{\boldsymbol{W}} \quad (\boldsymbol{Y} - \boldsymbol{H}\boldsymbol{W})^T(\boldsymbol{Y} - \boldsymbol{H}\boldsymbol{W})$$

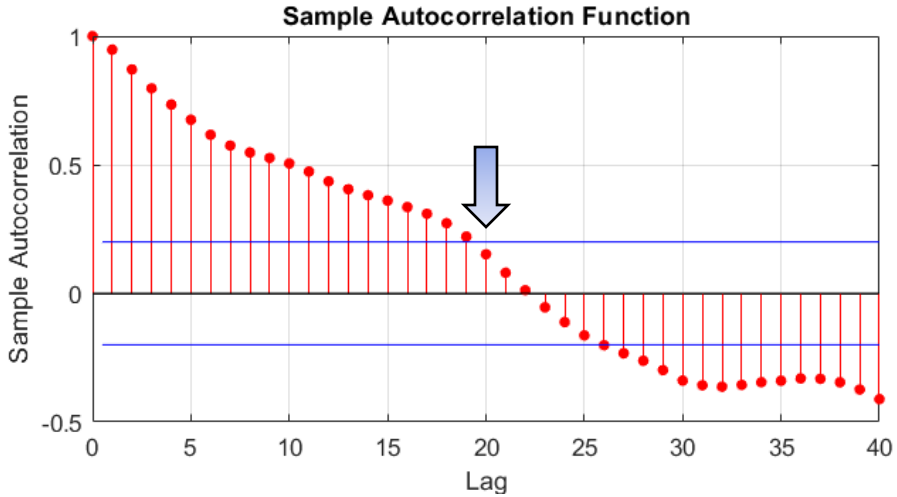
The solution:

$$\boldsymbol{W} = (\boldsymbol{H}^T\boldsymbol{H})^{-1}\boldsymbol{H}^T\boldsymbol{Y}$$

How to find the no. of lags, p?

Check at which number of lags will the autocorrelation fall below the threshold.

```
>> autocorr(y,40)
```



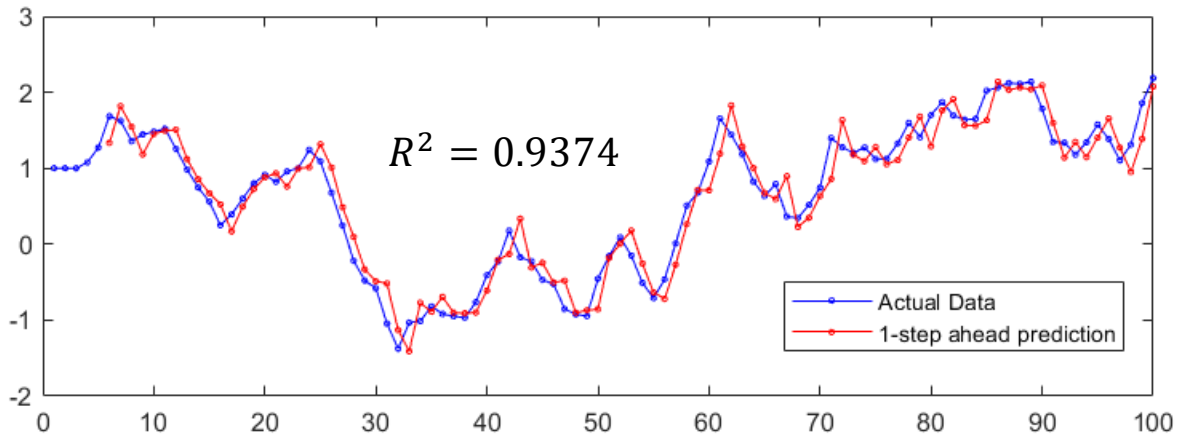
© 2023 Karl Ezra S. Pilario, PhD.

Generalized Linear Models

Example:

Fit an AR(5) model to the following $y(k)$ data:

```
% Data Set
y = [1 1 1 1.0771 1.27 1.6855 1.6244 1.3567 1.445 1.4815 ...
     1.5192 1.2542 0.9811 0.7452 0.5597 0.2493 0.3923 0.5991 ...
     0.8004 0.9127 0.8207 0.9581 0.9999 1.2415 1.0918 0.6764 ...
     0.2462 -0.219 -0.4789 -0.581 -1.0453 -1.3763 -1.033 ...
     -1.0118 -0.8202 -0.9176 -0.9497 -0.9665 -0.7633 -0.4064...
     -0.2284 0.1766 -0.1724 -0.2264 -0.4699 -0.5254 -0.853 ...
     -0.9229 -0.946 -0.4528 -0.1557 0.0904 -0.1513 -0.5082 ...
     -0.7096 -0.4628 0.009 0.5079 0.6805 1.0877 1.6542 1.4418 ...
     1.1844 0.8242 0.6357 0.7908 0.3636 0.3437 0.5172 0.7446 ...
     1.3979 1.276 1.2013 1.2746 1.1203 1.1233 1.3288 1.5964 ...
     1.4091 1.7006 1.8727 1.6957 1.6384 1.6498 2.0206 2.0632 ...
     2.1222 2.11 2.1374 1.7848 1.3462 1.3377 1.1796 1.3423 ...
     1.5751 1.3838 1.1046 1.311 1.8586 2.1878];
```



Solution:

```
% Data Set
y = [1 1 1 1.0771 1.27 1.6855 1.6244 1.3567 1.445 1.4815 ...
     1.5192 1.2542 0.9811 0.7452 0.5597 0.2493 0.3923 0.5991 ...
     0.8004 0.9127 0.8207 0.9581 0.9999 1.2415 1.0918 0.6764 ...
     0.2462 -0.219 -0.4789 -0.581 -1.0453 -1.3763 -1.033 ...
     -1.0118 -0.8202 -0.9176 -0.9497 -0.9665 -0.7633 -0.4064...
     -0.2284 0.1766 -0.1724 -0.2264 -0.4699 -0.5254 -0.853 ...
     -0.9229 -0.946 -0.4528 -0.1557 0.0904 -0.1513 -0.5082 ...
     -0.7096 -0.4628 0.009 0.5079 0.6805 1.0877 1.6542 1.4418 ...
     1.1844 0.8242 0.6357 0.7908 0.3636 0.3437 0.5172 0.7446 ...
     1.3979 1.276 1.2013 1.2746 1.1203 1.1233 1.3288 1.5964 ...
     1.4091 1.7006 1.8727 1.6957 1.6384 1.6498 2.0206 2.0632 ...
     2.1222 2.11 2.1374 1.7848 1.3462 1.3377 1.1796 1.3423 ...
     1.5751 1.3838 1.1046 1.311 1.8586 2.1878];
N = length(y);                          % No. of samples of y
p = 5;                                  % No. of lags in the AR model
H = y(hankel(1:p,p:(N-1)))';            % Create Hankel matrix
H = fliplr([H ones(length(H),1)]);      % Introduce bias (ones) in H
Y = y((p+1):N)';                        % Create target vector

% Get parameters by least-squares
W = (H'*H)\(H'*Y); disp(W);

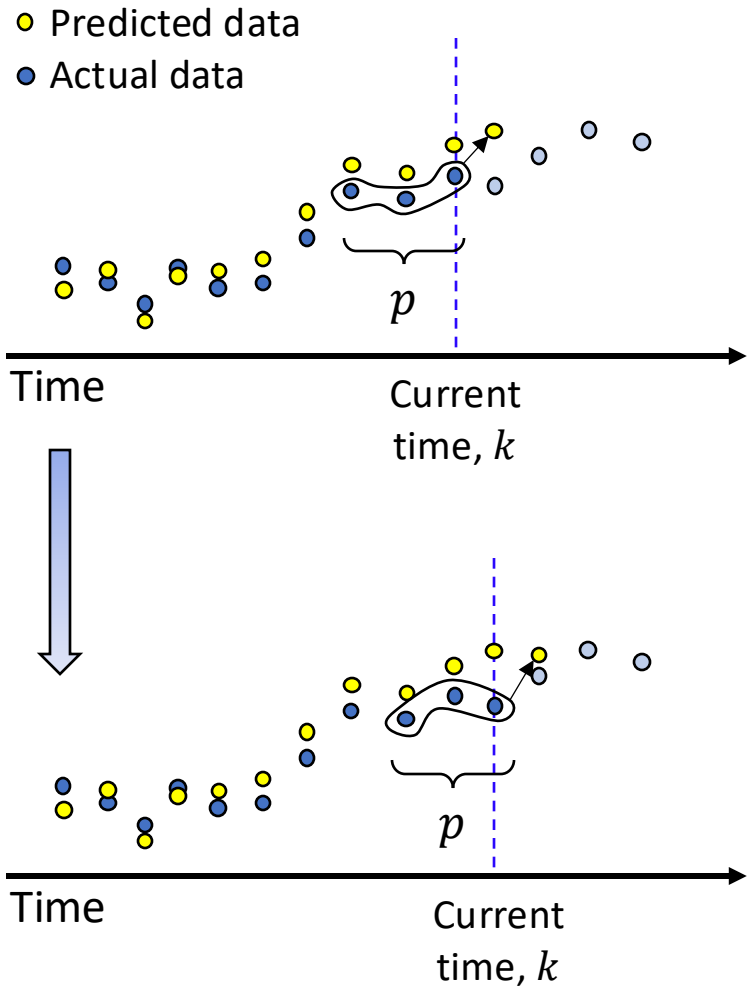
% Test the model for 1-step ahead prediction
ypred = zeros(1,N); ypred(1:p) = y(1:p);
for j = (p+1):N
    ypred(j) = W'*[1; y(j-(1:p))'];
end

% Compare the predicted to actual
plot(1:N,y,'ob-','MarkerSize',2); hold on;
plot((p+1):N,ypred((p+1):N),'or-','MarkerSize',2);
legend('Actual Data','1-step ahead prediction'); hold off;
R2 = 1 - sum((ypred-y).^2)/sum((y - mean(y)).^2);
fprintf('R^2: %.4f\n',R2);
```

© 2023 Karl Ezra S. Pilario, PhD.

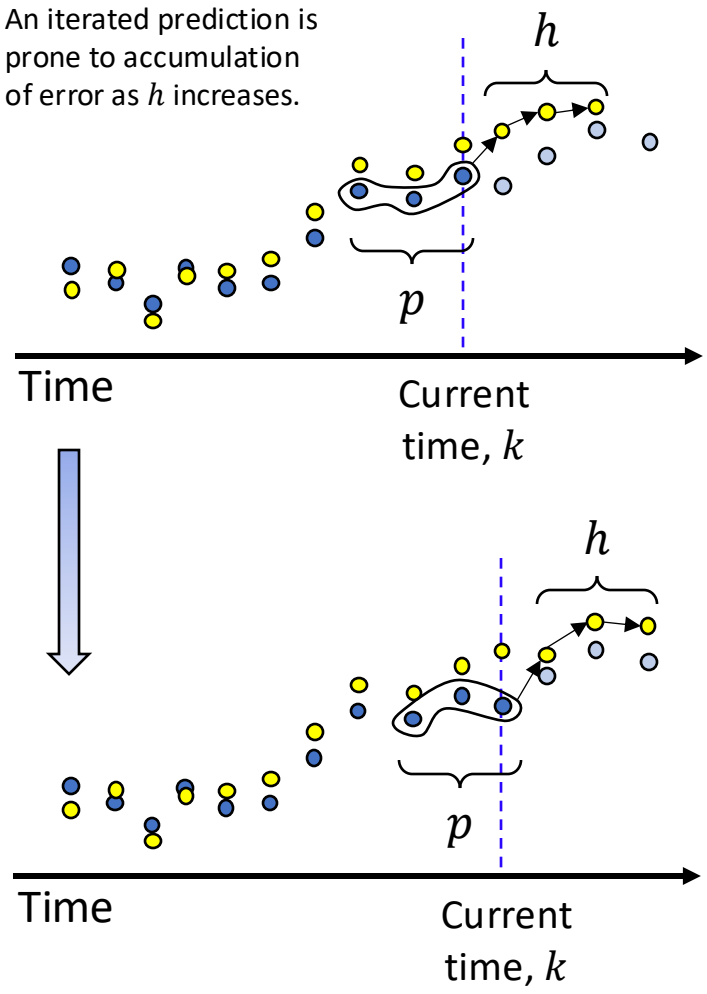
Model Testing: Prediction vs. Simulation

1-step ahead prediction



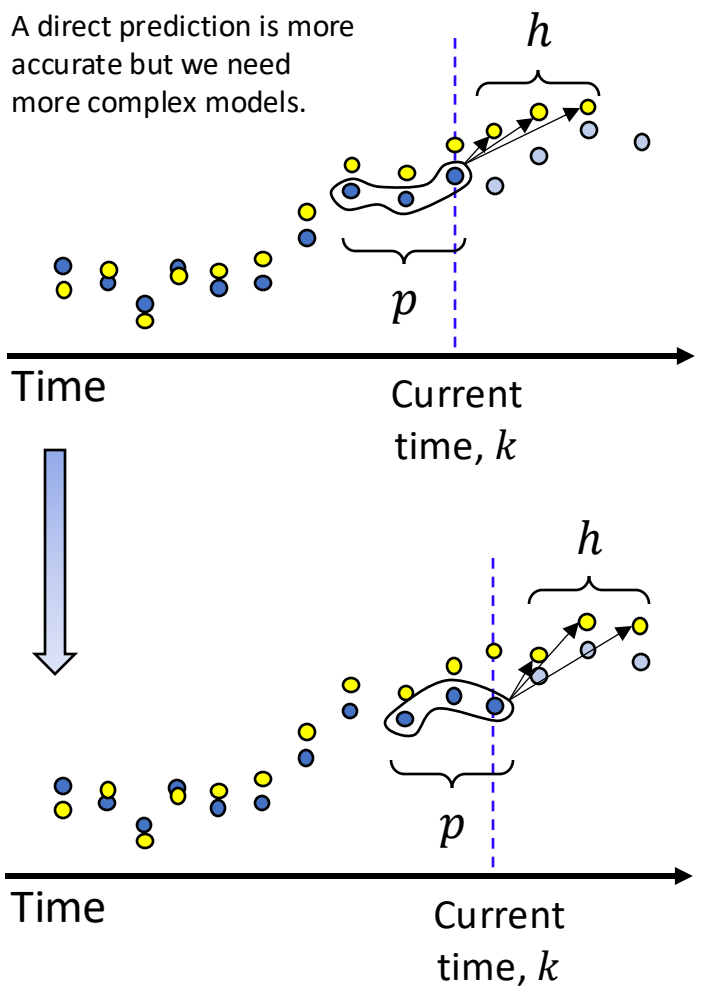
h-step ahead prediction (**iterated**)

An iterated prediction is prone to accumulation of error as h increases.



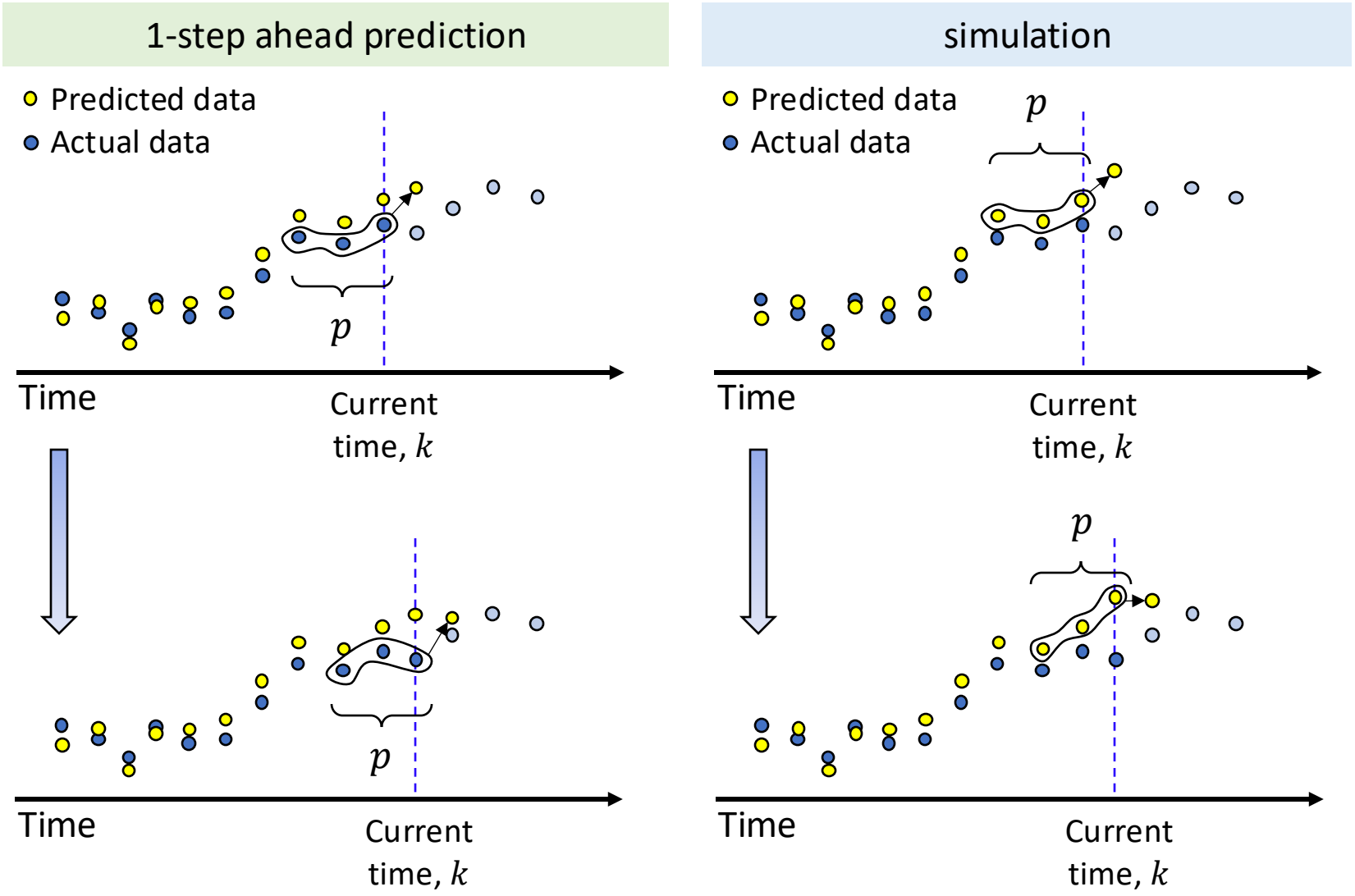
h-step ahead prediction (**direct**)

A direct prediction is more accurate but we need more complex models.



© 2023 Karl Ezra S. Pilario, PhD.

Model Testing: Prediction vs. Simulation



- In simulation, the model produces predictions without any actual data as input to the model.
- Simulation is the same as an infinite-step ahead prediction.
- It is more difficult to find a model that performs well for simulation than to find one that performs well for h-step ahead prediction.
- In simulation, one also needs to ensure that the model is stable. In prediction, there is no need.
- Simulation tasks are more suited for process optimization, control, and fault detection.

© 2023 Karl Ezra S. Pilario, PhD.

Generalized Linear Models

Summary of Generalized Linear Models:

Name	Model	
MA (Moving average)	$A(z)y(k) = \frac{B(z)}{F(z)}u(k) + \frac{\boldsymbol{C(z)}}{D(z)}e(k)$	C
AR (Autoregressive)	$\boldsymbol{A(z)}y(k) = \frac{B(z)}{F(z)}u(k) + \frac{C(z)}{D(z)}e(k)$	A
ARX (Autoregressive, exogeneous)	$\boldsymbol{A(z)}y(k) = \frac{\boldsymbol{B(z)}}{F(z)}u(k) + \frac{C(z)}{D(z)}e(k)$	AB
ARMA (Autoregressive, moving average)	$\boldsymbol{A(z)}y(k) = \frac{B(z)}{F(z)}u(k) + \frac{\boldsymbol{C(z)}}{D(z)}e(k)$	AC
ARMAX (Autoregressive, moving average, exogeneous)	$\boldsymbol{A(z)}y(k) = \frac{\boldsymbol{B(z)}}{F(z)}u(k) + \frac{\boldsymbol{C(z)}}{D(z)}e(k)$	ABC
TRANSFER FUNC. or OUTPUT-ERROR	$A(z)y(k) = \frac{\boldsymbol{B(z)}}{\boldsymbol{F(z)}}u(k) + \frac{C(z)}{D(z)}e(k)$	BF
ARARX (Autoregressive y, Autoregressive e, exogeneous)	$\boldsymbol{A(z)}y(k) = \frac{\boldsymbol{B(z)}}{F(z)}u(k) + \frac{C(z)}{\boldsymbol{D(z)}}e(k)$	ABD
ARARMAX (Autoreg y, Autoreg e, moving average, exogeneous)	$\boldsymbol{A(z)}y(k) = \frac{\boldsymbol{B(z)}}{F(z)}u(k) + \frac{\boldsymbol{C(z)}}{\boldsymbol{D(z)}}e(k)$	ABCD
BOX-JENKINS	$A(z)y(k) = \frac{\boldsymbol{B(z)}}{\boldsymbol{F(z)}}u(k) + \frac{\boldsymbol{C(z)}}{\boldsymbol{D(z)}}e(k)$	BCDF
FIR (Finite Impulse Response)	$A(z)y(k) = \frac{\boldsymbol{B(z)}}{F(z)}u(k) + \frac{C(z)}{D(z)}e(k)$	B

- Usually, once the parameters (a_i, b_i, ...) are solved, we don't give them a physical interpretation. The models are treated as a black-box.
- The generalized linear models are simple enough to be trained using least-squares method, without iterations, because they are only linear.
- A requirement for these models to work is that the data itself is stationary (has no overall trend). We can try differencing if the time series is non-stationary. (e.g. use ARIMA)

© 2023 Karl Ezra S. Pilario, PhD.

Outline

- Recall: System Identification

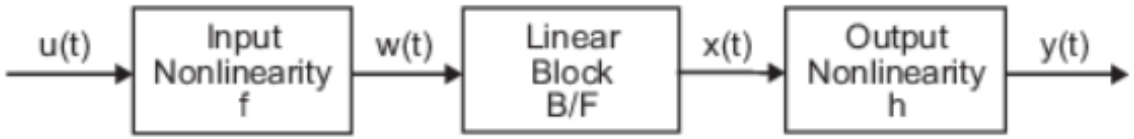
- General Linear Model Structure
  - SISO Models: AR, ARX, ARMA, ARMAX
  - N-step Ahead Prediction vs. Simulation

- **Nonlinear Model Structures**
  - Hammerstein-Wiener Process
  - Recurrent Neural Networks in MATLAB
  - Application to Simulated Data

© 2023 Karl Ezra S. Pilario, PhD.

Hammerstein-Wiener Process

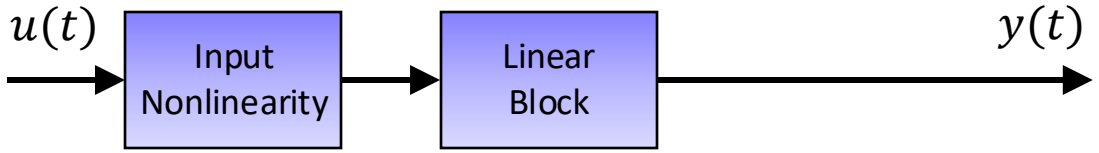
- The model consists of 3 blocks:



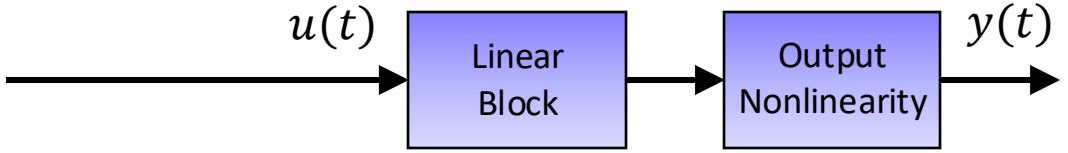
$$A(z)y(k) = \frac{B(z)}{F(z)}u(k) + \frac{C(z)}{D(z)}e(k)$$

- The input/output nonlinearities are only static.
- The linear block handles the internal dynamics.

Hammerstein Model



Wiener Model



© 2023 Karl Ezra S. Pilario, PhD.

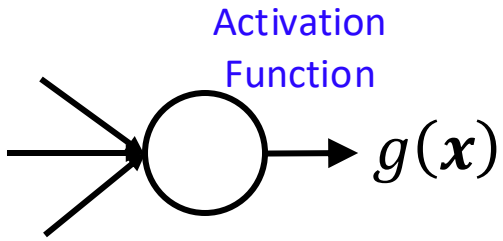
Neural Networks

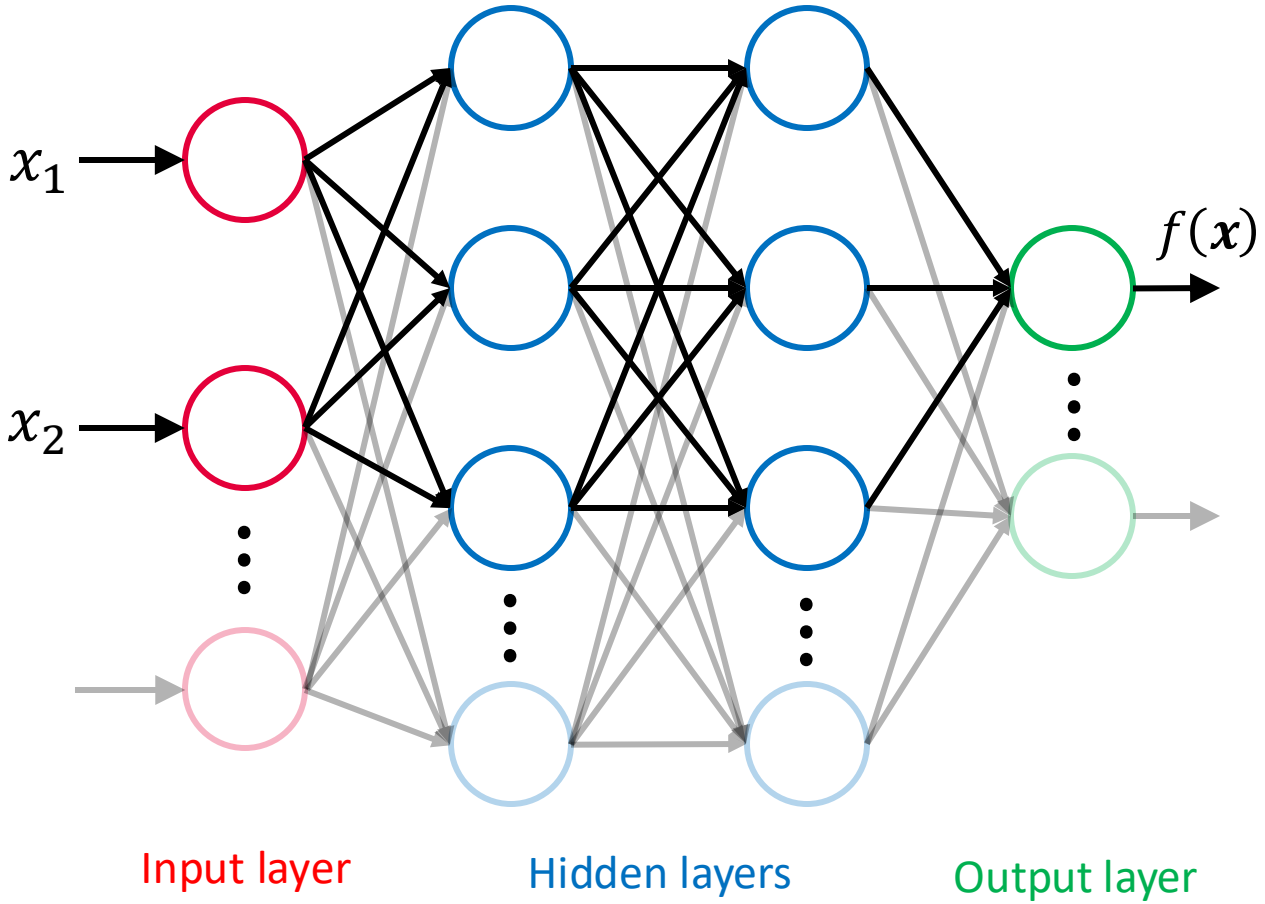
- Neural networks are inspired from the structure of the human brain.
- The model consists of neurons that pass information from one layer to the next.
- It was theoretically proven that neural networks have a universal approximation property:

> They can theoretically approximate any function to an arbitrary degree of accuracy. (*Hornik et al., 1989*)

Each neuron is a function,

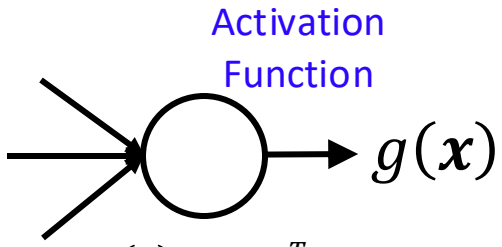
$$g(\boldsymbol{x}) = \sigma(\boldsymbol{w}^T\boldsymbol{x} + b)$$





© 2023 Karl Ezra S. Pilario, PhD.

Neural Networks



$$g(\boldsymbol{x}) = \sigma(\boldsymbol{w}^T\boldsymbol{x} + b)$$

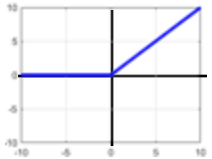
- Different choices of activation function:

Linear



$$\sigma(z) = z$$

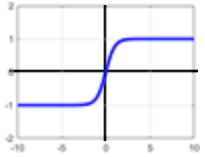
ReLU (Rectified Linear Unit)



$$\sigma(z) = \begin{cases} z, & z > 0 \\ 0, & z \leq 0 \end{cases} \text{ or}$$

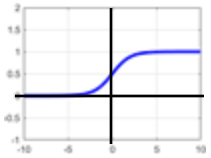
$$\sigma(z) = \max(0, z)$$

Tanh (tansig)

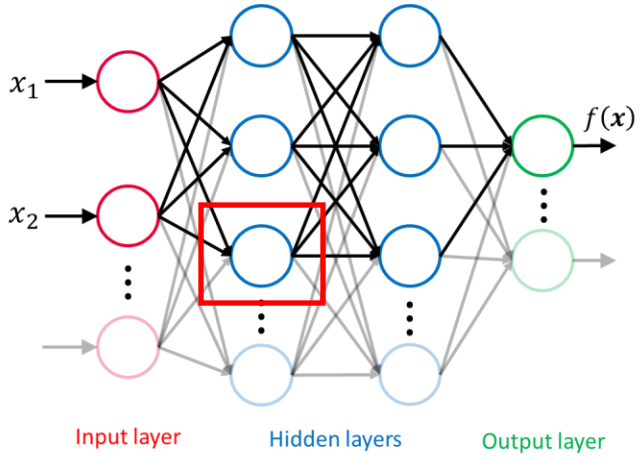


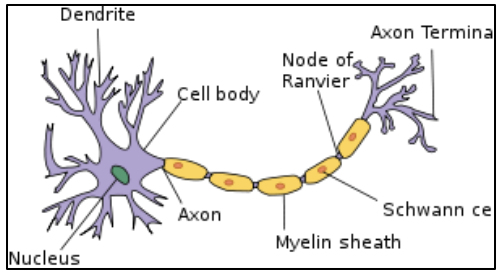
$$\sigma(z) = \frac{2}{1 + \exp(-2z)} - 1$$

Sigmoid

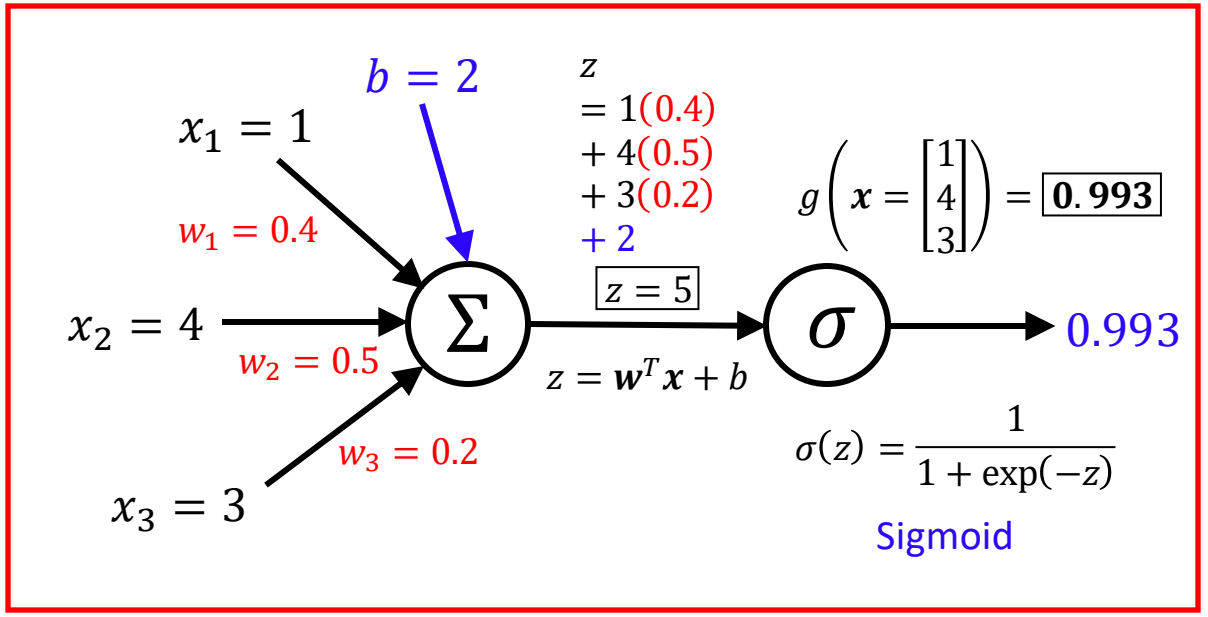


$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$





Perceptron (McCulloch-Pitts, 1943)

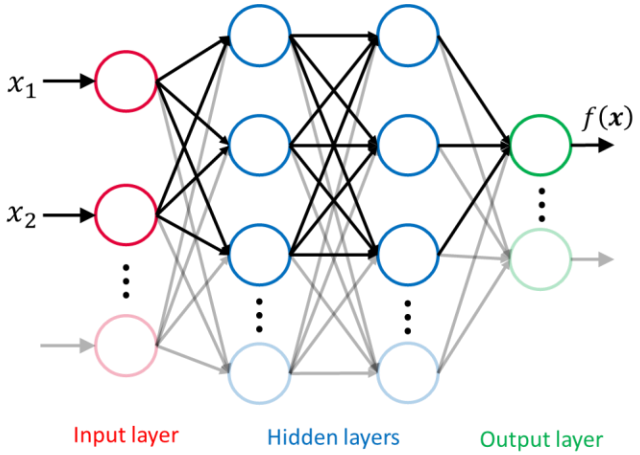


© 2023 Karl Ezra S. Pilario, PhD.

Neural Networks

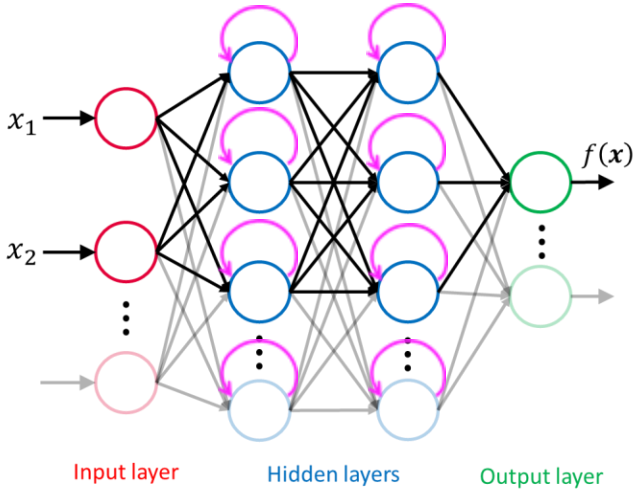
Feedforward Network (FFN)

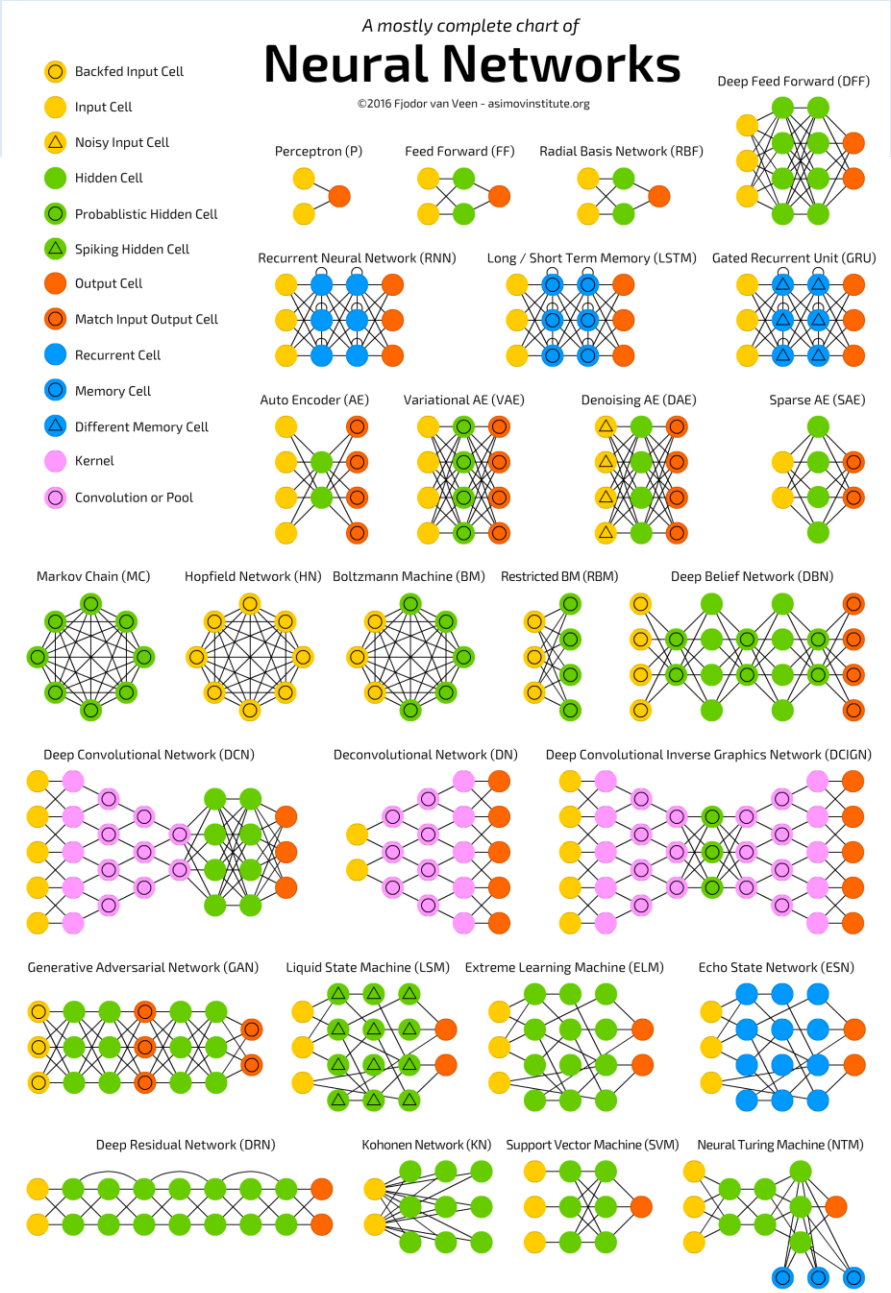
All flow of information is from inputs to outputs only.



Recurrent Neural Network

The hidden neurons are recurrent cells, where the information is internally fed back from output to input.



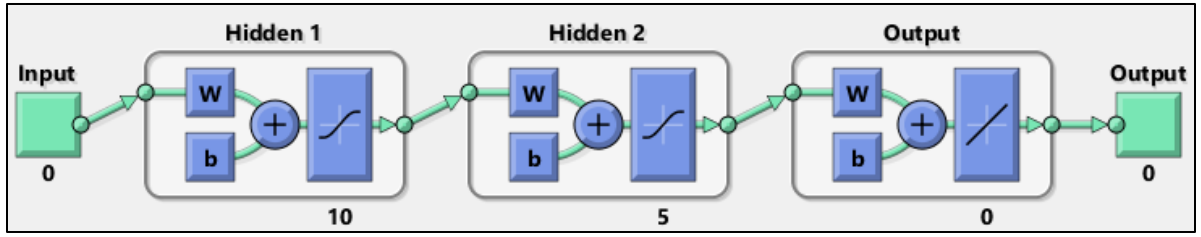


https://towardsdatascience.com/the-mostly-complete-chart-of-neural-networks-explained-3fb6f2367464

© 2023 Karl Ezra S. Pilario, PhD.

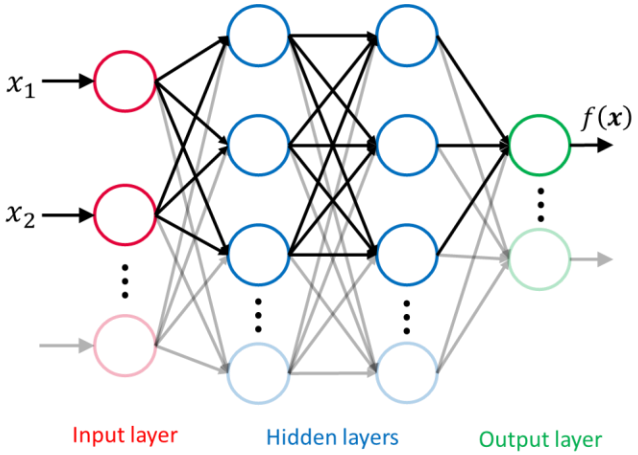
Neural Networks

MATLAB FEEDFORWARD NET MODEL

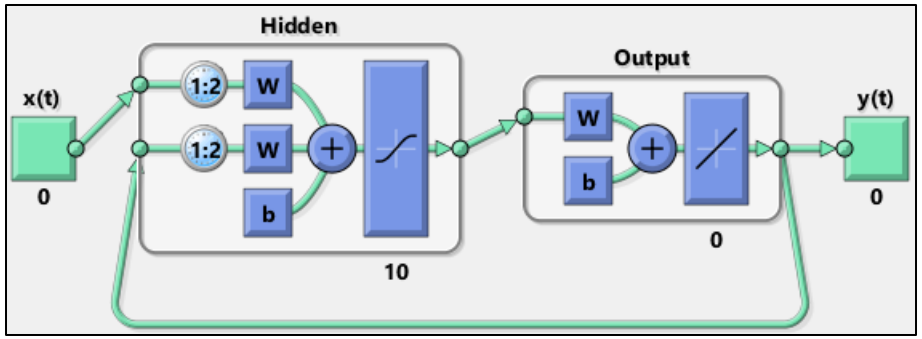


MATLAB Code:

```
>> net=feedforwardnet([10 5]);
>> view(net)
```

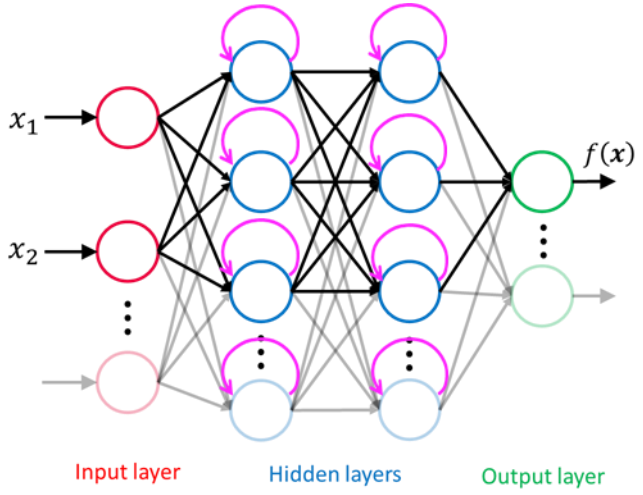


MATLAB CLOSED-LOOP NARX MODEL



MATLAB Code:

```
>> net=closeloop(narxnet(1:2,1:2,10));
>> view(net)
```



© 2023 Karl Ezra S. Pilario, PhD.

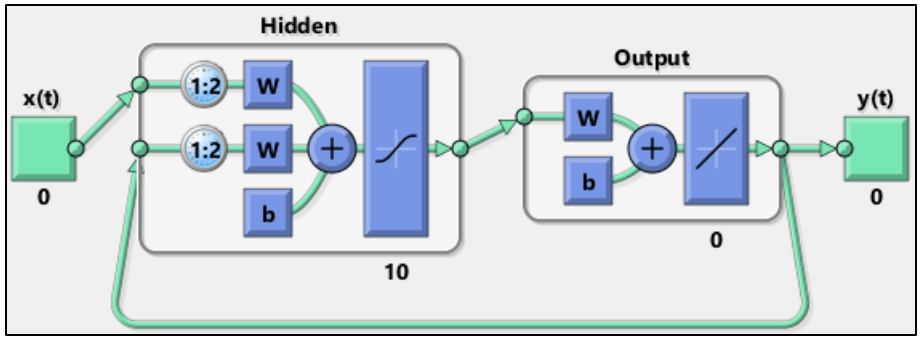
Neural Networks

Neural Network Training

Once the neural network architecture is defined, we can train it using various *gradient descent* algorithms:

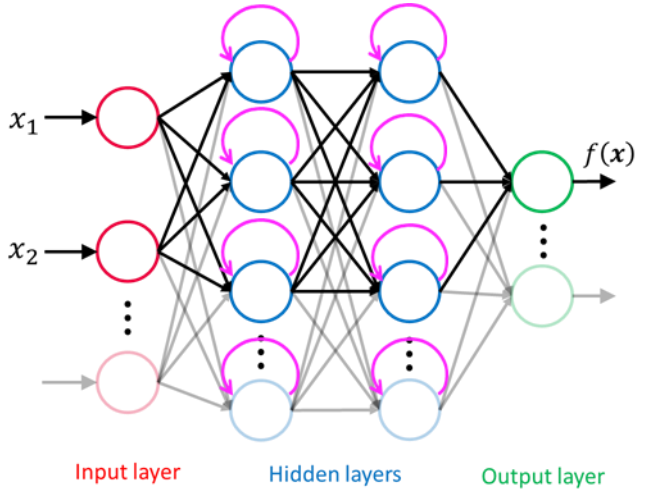
'trainlm'	Levenberg-Marquardt
'trainbr'	Bayesian Regularization
'trainbfg'	BFGS Quasi-Newton
'trainrp'	Resilient Backpropagation
'trainscg'	Scaled Conjugate Gradient
'traincgb'	Conjugate Gradient with Powell/Beale Restarts
'traincgf'	Fletcher-Powell Conjugate Gradient
'traincgp'	Polak-Ribiére Conjugate Gradient
'trainoss'	One Step Secant
'traingdx'	Variable Learning Rate Gradient Descent
'traingdm'	Gradient Descent with Momentum
'traingd'	Gradient Descent

MATLAB CLOSED-LOOP NARX MODEL



MATLAB Code:

```
>> net=closeloop(narxnet(1:2,1:2,10));
>> view(net)
```

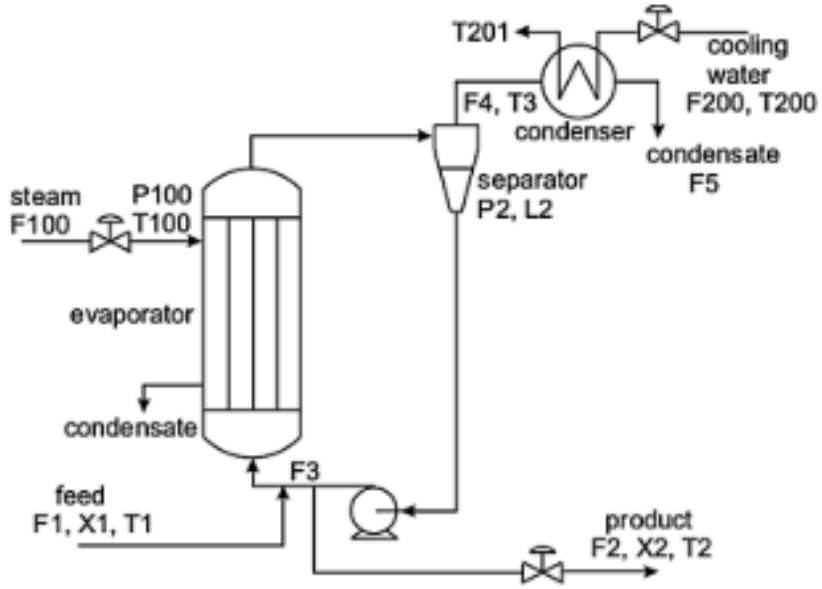


© 2023 Karl Ezra S. Pilario, PhD.

Neural Networks

Example:

Newell-Lee Evaporator System (Newell, Lee, 1989)



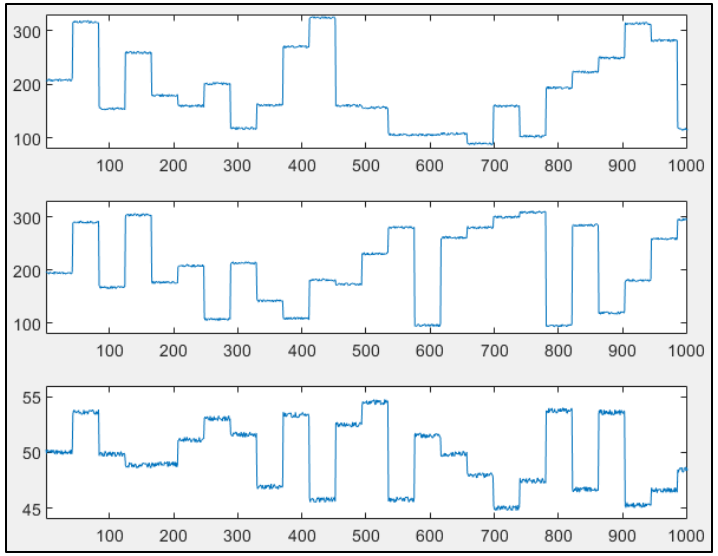
- Boils away the liquor from feed to produce a concentrate using a forced-circulation evaporator.
- Uses steam and cooling water utilities.
- There is one control loop for controlling the level in the separator by manipulating the product valve.
- Highly nonlinear system.

Inputs:

F200

P100

F3

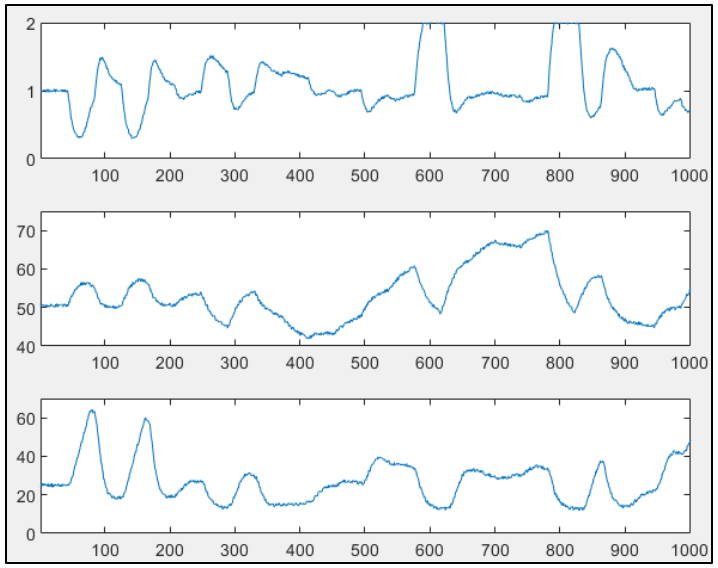


Outputs:

L2

P2

X2

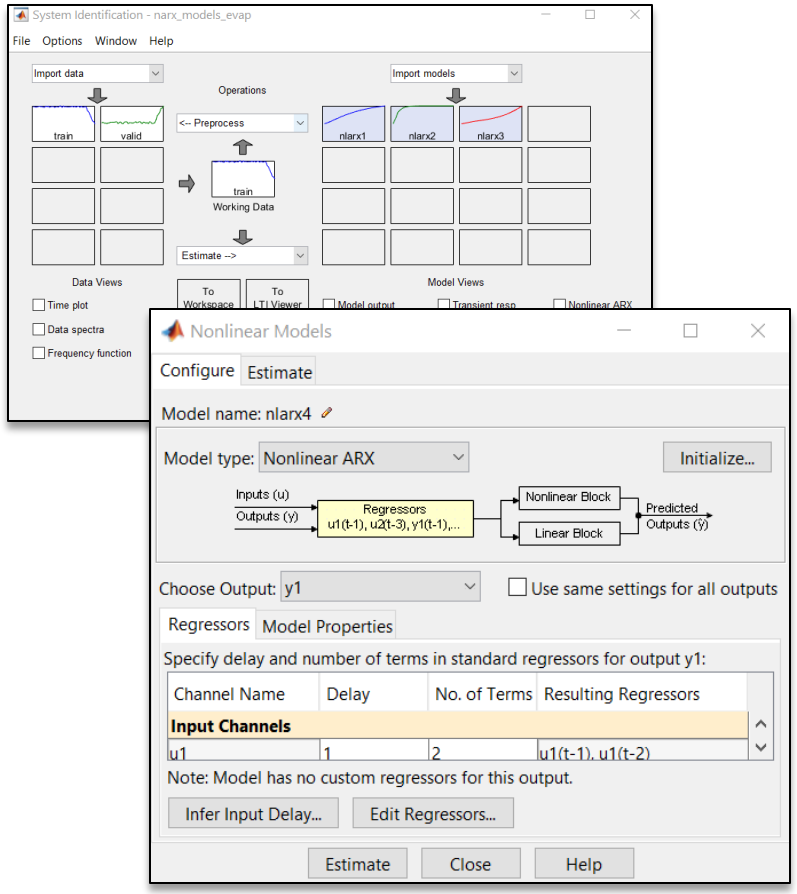


© 2023 Karl Ezra S. Pilario, PhD.

Neural Networks

There are three ways to use NARX in MATLAB:

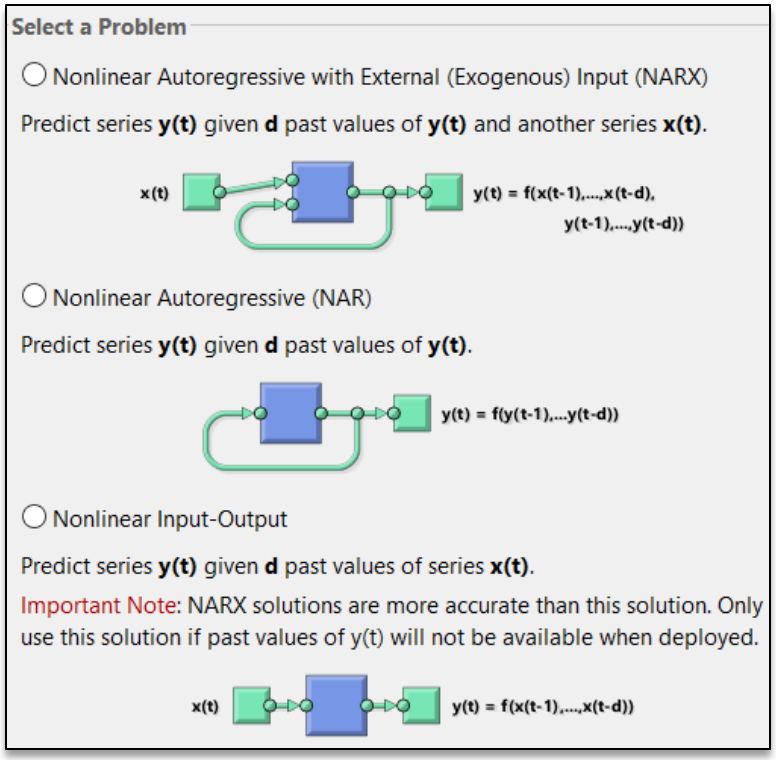
Using System Identification Toolbox



Using built-in function NARXNET

```
[X,T] = simplenarx_dataset;
net = narxnet(1:2,1:2,10);
view(net)
[Xs,Xi,Ai,Ts] = preparets(net,X,{},T);
net = train(net,Xs,Ts,Xi,Ai);
y = net(Xs,Xi,Ai);
```

Using Neural Net Time Series App



© 2023 Karl Ezra S. Pilario, PhD.

Outline

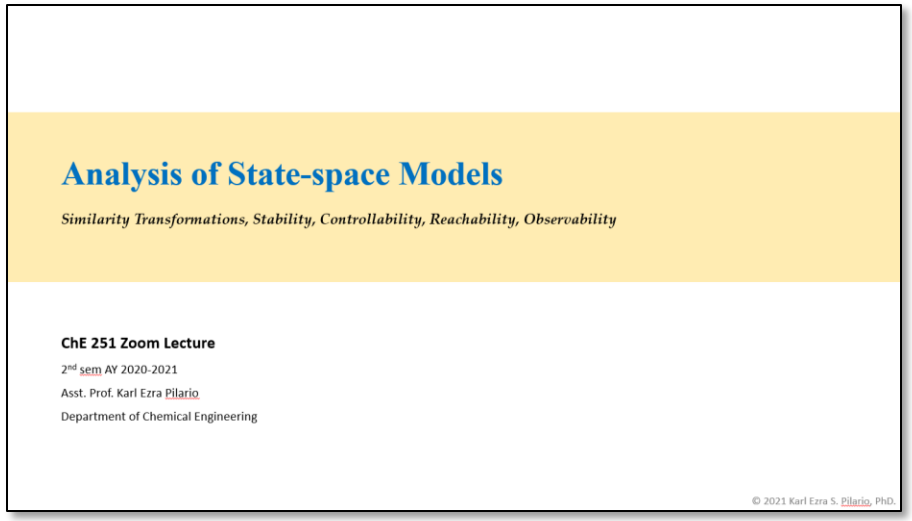
- Recall: System Identification

- General Linear Model Structure
  - SISO Models: AR, ARX, ARMA, ARMAX
  - N-step Ahead Prediction vs. Simulation

- Nonlinear Model Structures
  - Hammerstein-Wiener Process
  - Recurrent Neural Networks in MATLAB
  - Application to Simulated Data



Next Week:



© 2023 Karl Ezra S. Pilario, PhD.

MATLAB Scripts

Fig. 1 in Slide 6:

```
N = 100; y = zeros(1,N); y(1:3) = 1;
col = 'bmgrk'; close all; rng(1);
for k = 1:5
    for j = 4:N
        y(j) = 0.8*y(j-1) - 0.2*y(j-2)...
            + 0.3*y(j-3) + randn*0.1;
    end
    if k == 4
        plot(1:N,y,'-o','Color',col(k),...
            'MarkerSize',2,'LineWidth',2);
    else
        plot(1:N,y,'-o','Color',col(k),...
            'MarkerSize',2,'LineWidth',0.1);
    end
    hold on;
end
grid on;
```

Fig. 2 in Slide 6:

```
N = 100; y = zeros(1,N); y(1:3) = 1;
e = randn(size(y))*0.1;
col = 'bmgrk'; close all; rng(1);
for k = 1:5
    for j = 4:N
        e(j) = randn*0.1;
        y(j) = 0.9*y(j-1) + 0.1*y(j-2) + e(j) + 2*e(j-1);
    end
    if k == 4
        plot(1:N,y,'-o','Color',col(k),...
            'MarkerSize',2,'LineWidth',2);
    else
        plot(1:N,y,'-o','Color',col(k),...
            'MarkerSize',2,'LineWidth',0.1);
    end
    hold on;
end
grid on;
```

© 2023 Karl Ezra S. Pilario, PhD.

MATLAB Scripts

NARX for the Evaporator Data Set:

```
load evap_datasets train valid;

%% Convert data sets into cell arrays for NARX
% Let X1 = inputs (u) and T1 = outputs (y) of Training Set
X1 = tonndata(train(:,1:3),false,false);
T1 = tonndata(train(:,4:6),false,false);

% Let X2 = inputs (u) and T2 = outputs (y) of Training Set
X2 = tonndata(valid(:,1:3),false,false);
T2 = tonndata(valid(:,4:6),false,false);

%% Train a NARX using [X1,T1] (Training)
inputDelays = 1:2;                          % No. of lags in u
feedbackDelays = 1:2;                       % No. of lags in y
hiddenLayerSize = 12;                       % No. of hidden neurons

net = closeloop(narxnet(inputDelays,...
    feedbackDelays,hiddenLayerSize));       % Set architecture

% You can change how the data set is divided:
net.divideParam.trainRatio = 70/100;        % Training Data Size
net.divideParam.valRatio = 20/100;          % Validation Data Size
net.divideParam.testRatio = 10/100;         % Test Data Size

[x,xi,ai,t] = preparets(net,X1,{},T1);
net = trainbr(net,x,t,xi,ai);               % Set training algorithm

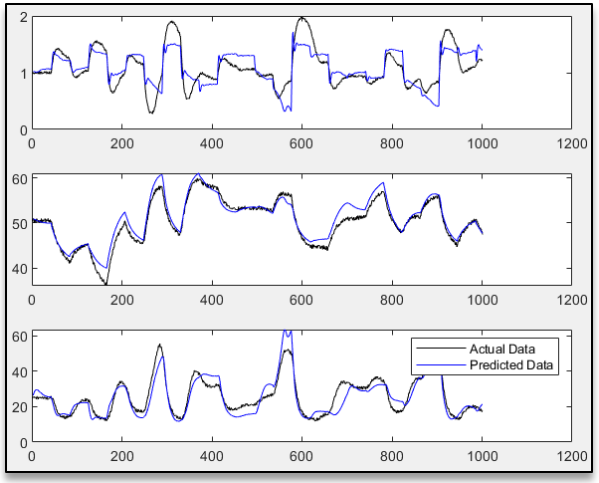
%% Test the NARX using [X2,T2] (Validation)
[x,xi,ai] = preparets(net,X2,{},T2);
Y = cell2mat(net(x,xi,ai))';
M = length(Y); L = length(valid);
fit = (1-goodnessOfFit(Y,valid((L-M+1):L,4:6),'NMSE'))*100;
```

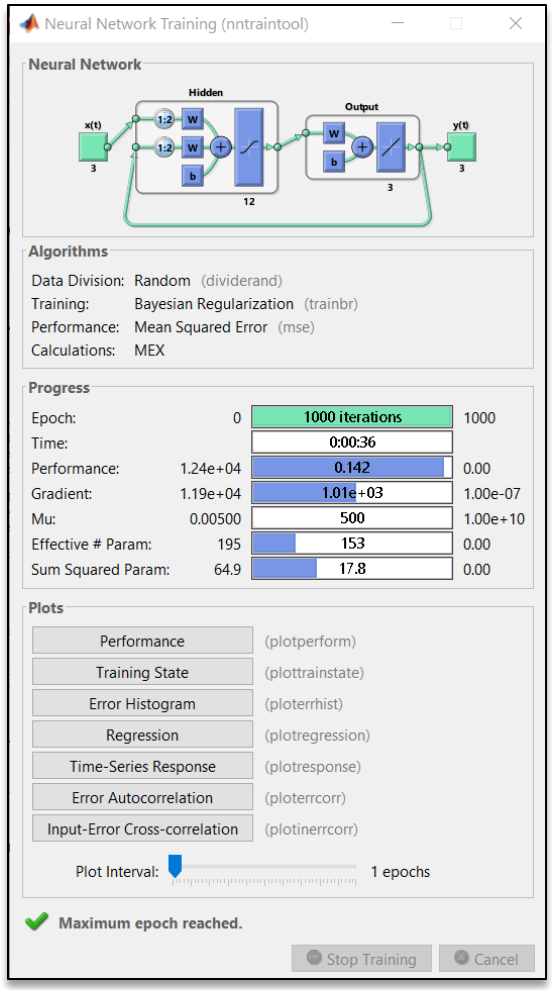
```
%% Plot the results (NARX vs. Actual)
subplot(311);
plot(1:L,valid(:,4),'k'); hold on;
plot((L-M+1):L,Y(:,1),'b'); hold off;

subplot(312);
plot(1:L,valid(:,5),'k'); hold on;
plot((L-M+1):L,Y(:,2),'b'); hold off;

subplot(313);
plot(1:L,valid(:,6),'k'); hold on;
plot((L-M+1):L,Y(:,3),'b'); hold off;
legend('Actual Data','Predicted Data');

ylab = {'L2','P2','X2'};
for j = 1:length(fit)
    fprintf('Fit %s:\tR^2 = %.4f%%\n',ylab{j},fit(j));
end
fprintf('Average fit: R^2 = %.4f%%\n',mean(fit));
```





© 2023 Karl Ezra S. Pilario, PhD.